```
      val restartSettings = RestartSettings(     ❶
        minBackoff = 1.seconds,
        maxBackoff = 3.seconds,
        randomFactor = 0.2).withMaxRestarts(30, 3.minutes)

      val restartSource = RestartSource.withBackoff(restartSettings) { () =>
        Source.futureSource {
          val responseStream: Source[Location, NotUsed] =
            client.follow()                      ❷
          Future(responseStream)
        }
      }
  ...
}
```

❶ Restart settings

❷ RPC client creates a source

The `minBackoff` is the initial time before the source tries again. If it fails, another second is added each time until the `maxBackoff` is reached. The `.withMaxRestarts` option sets two values, where the first of the two values reached becomes the end of the attempts. The first number is the absolute number of retries, and the second is the duration since the first fail. The `randomFactor` prevents multiple possible clients of the service from bouncing back at the same time. Here you have a deviation of 20% over the time of each retry.

The RPC call doesn't have input, and the response is a `Location`:

```
rpc  Follow(google.protobuf.Empty) returns (stream Location) {}
```

The complete test is shown in the next listing.

Listing 16.9 `RestartSource` consuming a gRPC service for 90 seconds